

Rigorous Analysis of the Erdős-Rado Sunflower Theorem

Charles EDOU NZE*

August 15, 2026

Abstract

We present a rigorous investigation into the Erdős-Rado Sunflower Theorem, a fundamental result in extremal combinatorics. A sunflower with w petals is a collection of w sets such that every pair of sets has the same intersection. We formalize the problem through strict axiomatic definitions, analyze the underlying combinatorial structures, and detail the classic inductive bounding arguments established by Erdős and Rado. Furthermore, we construct an explicit scaffolding for autoformalization in Lean 4 to ensure symbolic verification.

1 Axiomatic Definitions and Problem Statement

Definition 1.1. Let $\mathcal{F}$ be a family of sets. A subsystem $\mathcal{S} \subseteq \mathcal{F}$ consisting of w distinct sets $S_1, S_2, \dots, S_w$ is called a **sunflower** (or Δ -system) with w petals if there exists a core set C such that for all $1 \leq i < j \leq w$:

$$S_i \cap S_j = C \tag{1}$$

The sets $S_i \setminus C$ are disjoint and non-empty for $i = 1, \dots, w$ (if $C \neq S_i$).

Definition 1.2. The Erdős-Rado Sunflower Theorem asserts that there exists a function $f(k, w)$ such that any family $\mathcal{F}$ of sets, each of cardinality at most k , must contain a sunflower with w petals if $|\mathcal{F}| > f(k, w)$.

2 Contextual Literature

The foundational work on sunflowers was introduced by Erdős and Rado in 1960. They demonstrated that $f(k, w) \leq k!(w - 1)^k$. Recent literature explores robust sunflower lemmas and improvements on these upper bounds, continuing to rely on intersection properties of set systems. The classical proof utilizes induction on the size of the sets in the family, a technique analogous to applying Dirichlet's pigeonhole principle across bounded domain sizes.

*chercheur indépendant

3 Lemmas and Step-by-Step Proofs

3.1 Base Case for Induction

Lemma 3.1. *For $k = 1$ and any $w \geq 2$, $f(1, w) \leq w - 1$.*

Proof. Let $\mathcal{F}$ be a family of sets where each set has cardinality at most 1. Thus, every set in $\mathcal{F}$ is either a singleton or the empty set. Assume $|\mathcal{F}| > w - 1$. Since all sets in $\mathcal{F}$ are distinct, at most one set can be the empty set, and the rest must be distinct singleton sets. Any two distinct singleton sets (or a singleton and the empty set) are disjoint. Therefore, their intersection is the empty set $C = \emptyset$. If we select w sets from $\mathcal{F}$ (which is possible since $|\mathcal{F}| \geq w$), they satisfy equation (1) with $C = \emptyset$. Hence, they form a sunflower with w petals. $\square$

3.2 Inductive Step: The Core Argument

Lemma 3.2. *If the theorem holds for sets of size up to $k - 1$, it holds for sets of size up to k , specifically bounded by $f(k, w) \leq k!(w - 1)^k$.*

Proof. Assume, for induction, that $f(k - 1, w) \leq (k - 1)!(w - 1)^{k-1}$. Let $\mathcal{F}$ be a family of sets of size at most k , such that $|\mathcal{F}| > k!(w - 1)^k$.

Let $\mathcal{D}$ be a maximal family of pairwise disjoint sets within $\mathcal{F}$. We consider two cases based on the cardinality of $\mathcal{D}$.

Case 1: $|\mathcal{D}| \geq w$. By definition of $\mathcal{D}$, the sets in $\mathcal{D}$ are pairwise disjoint. Therefore, for any two sets $A, B \in \mathcal{D}$, $A \cap B = \emptyset$. Choosing any w sets from $\mathcal{D}$ yields a sunflower with w petals and core $C = \emptyset$.

Case 2: $|\mathcal{D}| < w$. Let X be the union of all sets in $\mathcal{D}$. Since each set in $\mathcal{D}$ has cardinality at most k , the total number of elements in X is strictly bounded:

$$\begin{aligned} |X| &\leq |\mathcal{D}| \cdot k \\ |X| &\leq (w - 1)k \end{aligned} \tag{2}$$

Since $\mathcal{D}$ is maximal, no set in $\mathcal{F}$ can be disjoint from X . Otherwise, such a set could be added to $\mathcal{D}$, contradicting its maximality. Thus, every set $S \in \mathcal{F}$ contains at least one element $x \in X$.

By the pigeonhole principle, there must exist an element $x \in X$ that is contained in many sets of $\mathcal{F}$. Specifically, let $\mathcal{F}_x$ be the subfamily of sets containing x . Since $\mathcal{F} = \bigcup_{x \in X} \mathcal{F}_x$, we have:

$$\begin{aligned} \max_{x \in X} |\mathcal{F}_x| &\geq \frac{|\mathcal{F}|}{|X|} \\ &> \frac{k!(w - 1)^k}{(w - 1)k} \\ &= (k - 1)!(w - 1)^{k-1} \end{aligned} \tag{3}$$

Let x^* be an element achieving this maximum, and let $\mathcal{F}' = \{S \setminus \{x^*\} \mid S \in \mathcal{F}_{x^*}\}$. The family $\mathcal{F}'$ consists of sets of cardinality at most $k - 1$. Furthermore, $|\mathcal{F}'| = |\mathcal{F}_{x^*}| > (k - 1)!(w - 1)^{k-1}$.

By the inductive hypothesis, $\mathcal{F}'$ must contain a sunflower with w petals, say $\{S'_1, S'_2, \dots, S'_w\}$ with core C' . Adding the element x^* back to each set yields $S_i = S'_i \cup \{x^*\}$ for $i = 1, \dots, w$. These sets $S_i \in \mathcal{F}$ form a sunflower with core $C = C' \cup \{x^*\}$. $\square$

4 Architecture for Autoformalization

The formal verification of the aforementioned lemmas can be implemented in Lean 4. The foundational types are specified as follows:

```
import Mathlib.Data.Finset.Basic
import Mathlib.Data.Fintype.Basic

def IsSunflower {\alpha : Type*} (F : Finset (Finset \alpha)) (
  core : Finset \alpha) : Prop :=
  forall A B, A \in F -> B \in F -> A \neq B -> A \cap B =
    core

def HasSunflower {\alpha : Type*} (F : Finset (Finset \alpha)) (w
  : Nat) : Prop :=
  \exists S : Finset (Finset \alpha), S \subseteq F \land S.
    card = w \land \exists core, IsSunflower S core

theorem erdos_rado_base_case {\alpha : Type*} (F : Finset (Finset
  \alpha)) (w : Nat) (hw : w >= 2)
  (hsize : \forall A \in F, A.card <= 1) (hcard : F.card > w -
    1) :
  HasSunflower F w := by
  admit

theorem erdos_rado_induction {\alpha : Type*} (F : Finset (Finset
  \alpha)) (k w : Nat) (hw : w >= 2)
  (hsize : \forall A \in F, A.card <= k) (hcard : F.card > k.
    factorial * (w - 1)^k)
  (ih : \forall G : Finset (Finset \alpha), (\forall B \in G,
    B.card <= k - 1) \rightarrow G.card > (k - 1).factorial * (
    w - 1)^(k - 1) \rightarrow HasSunflower G w) :
  HasSunflower F w := by
  admit
```

This structure effectively bridges the informal combinatorial arguments to the rigorous symbolic requirements of Lean 4.